

Final Presentation

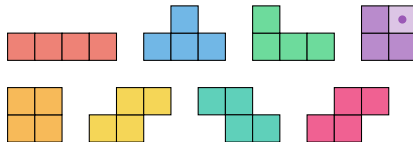
Knowledge Representation and Reasoning, 2026

Université de Mons (UMONS) — Faculté des Sciences

Generating and Solving 3D Tetracube Puzzles in Arbitrary Non-Rectangular Volumes with ASP

Authors: Abdelhadi Agourzam | Mohamed El ismaily

Supervisor: Jef Wijsen **Date:** May 2026



Part 1

The Problem & The Pieces

What is a Tetracube?

A 3D figure of **4 unit cubes** joined face-to-face.
8 distinct types $\Rightarrow 8 \times 4 = 32$ unit cubes total.

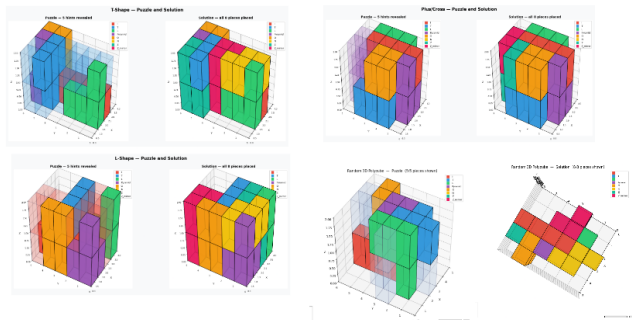
The Core Question

Can all 8 pieces pack into **non-rectangular** 3D volumes using a *single, unmodified* ASP engine?

Our Answer

Yes — any connected 32-cell volume works.
 One predicate: `cell(X,Y,Z)`.

Shapes solved: T-shape, L-shape,
 Plus/Cross, Random 2D & 3D polycubes



Puzzle & solution views for all tested shapes

C1. Shape-Agnostic ASP Engine

`puzzle.lp` has **zero shape-specific code**. Only `cell/3` changes per volume.

C2. Automated Rotation Generation

All **86 orientations** computed via 24 Python rotation matrices — no manual enumeration.

C3. Configurable Hint System

Parameter `num_hints` (0–7) **6 difficulty levels**.

C4. Independent Verification

`verify.py`: **6 checks** without trusting Clingo.

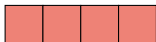
C5. Interactive 3D Visualiser

Matplotlib viewer with piece-by-piece reveal (`visualise.py`).

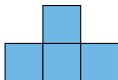
C6. Random Puzzle Generator

`random_puzzle.py`: solve any random 32-cell polycube automatically.

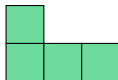
Key insight: swapping the shape file is the only change needed to solve a completely new 32-cell volume.



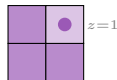
I (3 rot.)



T (12)



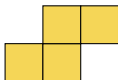
L (24)



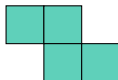
Pyramid (8)



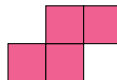
O (3)



N (12)



Z (12)



Z-mirror (12)

Total: 86 distinct orientations

7 planar + 1 non-planar (Pyramid)

Pyramid is the only truly 3D piece: 3 ground-floor cubes + 1 elevated cube at $z = 1$ (shown as filled dot).

How rotations are computed

Python applies each of the **24 rotation matrices** (the group S_4) to the base offsets (DX, DY, DZ), deduplicates, and exports to `tetracubes.lp`:

```
cube("I", 1, 0, 0, 0).  cube("I", 1, 0, 0, 1).
cube("I", 1, 0, 0, 2).  cube("I", 1, 0, 0, 3).
```

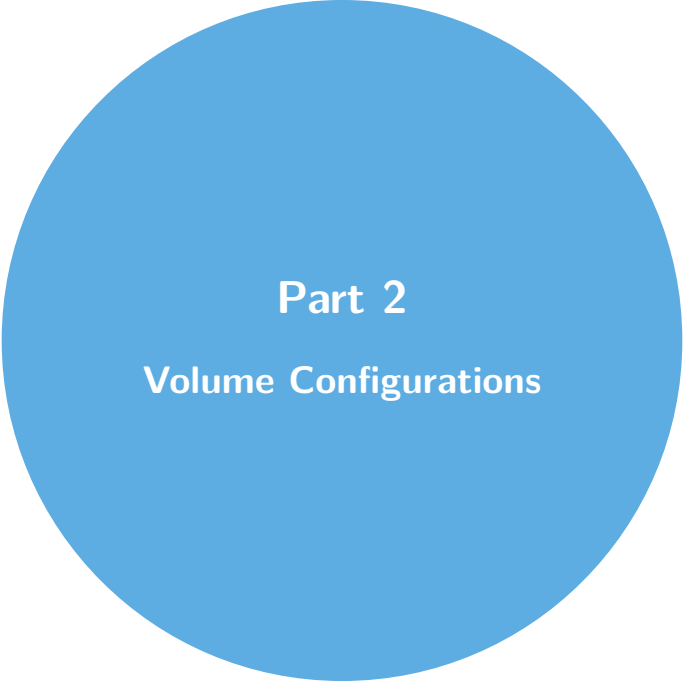
All 86 orientations generated this way

Rotation count by symmetry

Piece	Rot.	Why
I, O	3	High symmetry
T, N, Z, Zm	12	Partial sym.
Pyramid	8	Octahedral
L	24	No symmetry
Total	86	

Key fact

L has 24 orientations because it is *chiral and asymmetric* — every rotation matrix gives a new distinct placement. **Z** and **Z-mirror** are mirror images; neither is a rotation of the other.



Part 2

Volume Configurations

Old approach (hardcoded bounds)

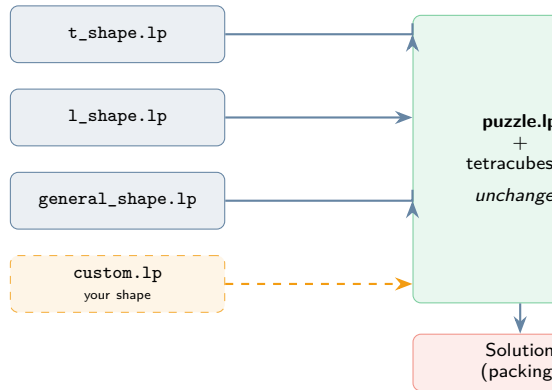
```
cell(X,Y,Z) :- X=0..3, Y=0..1, Z=0..3.
```

Only works for a $4 \times 2 \times 4$ box. Every new shape requires new solver code.

New approach: explicit cell/3 listing

```
% T-shape: bar + stem, depth 2
cell(X,Y,Z) :- X=0..3, Y=4..5, Z=0..1.
cell(X,Y,Z) :- X=1..2, Y=0..3, Z=0..1.
```

puzzle.lp only reads cell/3. Swap this file $\Rightarrow$ new shape, same solver.



Geometry: bar $\cup$ stem, depth 2

$$\underbrace{\{0..3\} \times \{4..5\}}_{\text{bar}} \cup \underbrace{\{1..2\} \times \{0..3\}}_{\text{stem}} \times \{0, 1\}$$

Cell count ✓

Bar: $4 \times 2 = 8/\text{layer}$ Stem: $2 \times 4 = 8/\text{layer}$

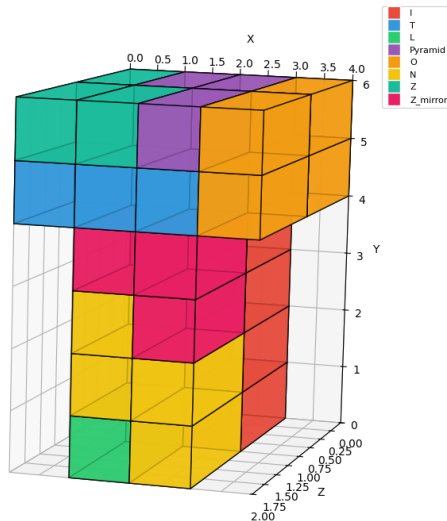
No overlap ($y_{\text{bar}} \geq 4$, $y_{\text{stem}} \leq 3$)

Total: $16 \times 2 = \mathbf{32}$

t_shape.lp

```
cell(X,Y,Z) :- X=0..3, Y=4..5, Z=0..1.
cell(X,Y,Z) :- X=1..2, Y=0..3, Z=0..1.
```

T-Shape — Solution (8/8 pieces shown)



Geometry: 4×5 minus 2×2 corner

$[0..3] \times [0..4] \setminus \{x \in \{2, 3\}, y \in \{3, 4\}\} \times \{0, 1\}$

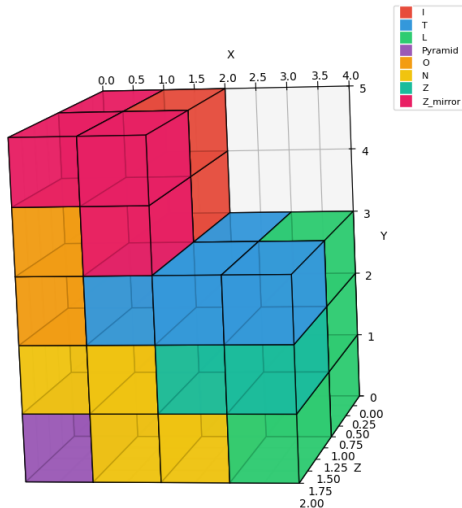
Non-convex boundary

This shape has a **concave corner**. The solver handles it identically — no change to the engine needed.

`l_shape.lp`

```
% Left columns: full height
cell(X,Y,Z) :- X=0..1, Y=0..4, Z=0..1.
% Right columns: bottom 3 rows only
cell(X,Y,Z) :- X=2..3, Y=0..2, Z=0..1.
```

L-Shape — Solution (8/8 pieces shown)



Geometry: body + arms, depth 2

$$\underbrace{\{0..3\} \times \{1..3\}}_{\text{body}} \cup \underbrace{\{1..2\} \times \{4\}}_{\text{top}} \cup \underbrace{\{1..2\} \times \{0\}}_{\text{bottom}} \times \{0..1\}$$

Cell count ✓

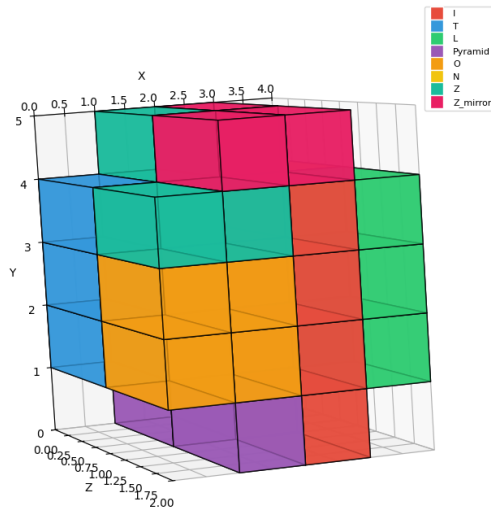
Body: 12/layer Arms: 4/layer

Total: $16 \times 2 = 32$

general_shape.lp

```
cell(X,Y,Z) :- X=0..3, Y=1..3, Z=0..1.
cell(X,Y,Z) :- X=1..2, Y=4..4, Z=0..1.
cell(X,Y,Z) :- X=1..2, Y=0..0, Z=0..1.
```

Plus/Cross Shape — Solution (8/8 pieces shown)



Part 3

ASP Modelling & Logic

The central design insight

The entire solver makes **one** assumption about the target: valid positions are ground atoms of `cell(X,Y,Z)`.

Logic and geometry are completely decoupled.

Key invariant:

`puzzle.lp` contains
zero references to specific
coordinates or shape dimensions.

Swap `cell/3` facts $\Rightarrow$
solver auto-adapts.

File	Responsibility
<code>*_shape.lp</code>	declares which cells exist
<code>puzzle.lp</code>	placement / coverage rules
<code>tetracubes.lp</code>	piece rotation offsets

```
# Solve T
clingo t_shape.lp puzzle.lp tetracubes.lp

# Solve Plus -- same engine!
clingo general_shape.lp puzzle.lp
      tetracubes.lp
```

Consequence

A new shape = a new `.lp` file. **Nothing else changes.**

Rule 1 — One placement per piece

```
1 { position(P,R,X,Y,Z)
  : cell(X,Y,Z),
    validOrientation(Type,R) } 1 :-
  tetracubeID(P), assignType(P,Type).
```

Exactly one anchor (X,Y,Z) + rotation R per piece. Anchor must be a valid cell.

Rule 2 — Boundary constraint

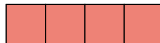
```
:- position(P,R,X,Y,Z),
   assignType(P,Type),
   cube(Type,R,DX,DY,DZ),
   not cell(X+DX, Y+DY, Z+DZ).
```

Every cell of a piece must be inside the cell/3 set. Pieces cannot "stick out".

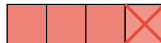
Deriving occupied cells

```
occupied(P, X+DX, Y+DY, Z+DZ) :-
  position(P,R,X,Y,Z),
  assignType(P,Type),
  cube(Type,R,DX,DY,DZ).
```

All 4 cells = anchor + offsets from tetracubes.lp.



✓ valid



✗ out of bounds

Rule 3 — Non-overlap

```
:- occupied(P1,X,Y,Z),
   occupied(P2,X,Y,Z),
   P1 != P2.
```

Two distinct pieces cannot share the same unit cube (X, Y, Z) .

Rule 4 — Full coverage

```
cellOccupied(X,Y,Z) :-
   occupied(P,X,Y,Z), tetracubeID(P).

:- cell(X,Y,Z),
   not cellOccupied(X,Y,Z).
```

Every cell declared by cell/3 must be covered by exactly one piece.

The four rules together

Rule	Guarantee
1	Each piece placed exactly once
2	No piece goes outside the shape
3	No two pieces share a cell
4	All 32 cells are covered

What Clingo does

Searches for an assignment satisfying all 4 rules simultaneously — a **perfect tiling**.

No solution exists $\Rightarrow$ UNSATISFIABLE.

Solution exists $\Rightarrow$ answer set = packing.

Hint generation in ASP

```
#const num_hints = 5.

% Select exactly num_hints pieces to reveal
{ hint(P) : tetracubeID(P) } num_hints.
:- num_hints != #count{P : hint(P)}.

% Bias toward diverse selection
#heuristic hint(P) : tetracubeID(P).
    [seed@3, false]
```

Override at runtime:

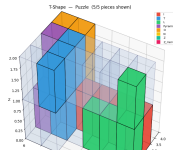
```
clingo ... -c num_hints=3
```

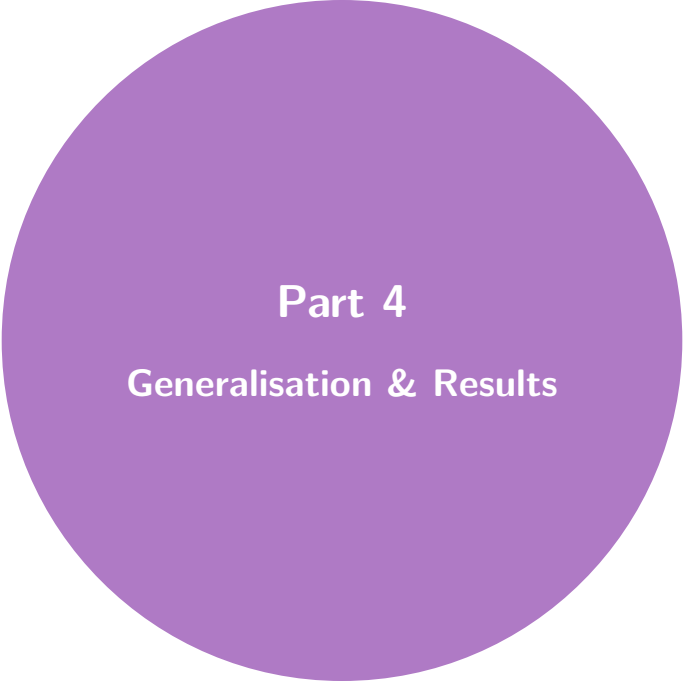
Difficulty levels

Hints	Hidden	Level
7	1	Very Easy
5-6	2-3	Easy
3-4	4-5	Medium
1-2	6-7	Hard
0	8	Harder

Hints = pre-placed pieces (given to player).

Hidden = pieces the player must place.





Part 4

Generalisation & Results

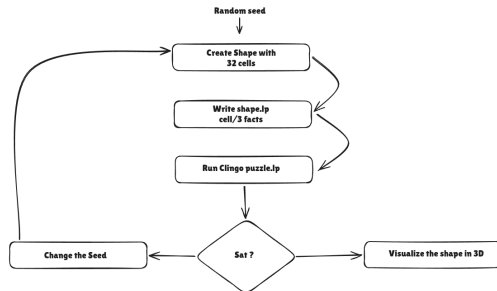
random_puzzle.py — Eden growth model

- 1 Start from one seed cell
- 2 Maintain a **frontier**: all face-neighbours of the current shape
- 3 Randomly pick one frontier cell, add it
- 4 Repeat until **32 cells** reached
- 5 Guarantees connectivity by construction

```
# Random 2D slab (default, reliable)
python random_puzzle.py
```

```
# Fixed seed, hard puzzle, ASCII art
python random_puzzle.py --seed 42 \
    --hints 2 --show-shape
```

```
# True 3D polycube
python random_puzzle.py --mode 3d
```



Two generation modes

Mode	Growth	Reliability
2D (default)	$\pm x, \pm y$, depth=2	High
3D (-mode 3d)	$\pm x, \pm y, \pm z$	Low-Med

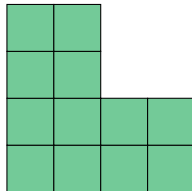
Why 2D is reliable

Depth-2 slabs give every cell ≥ 2 z-neighbours — enough room for any tetracube. Solve: < 30 ms.

Why 3D can fail

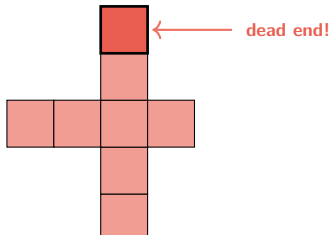
Eden 3D creates **thin tendrils**: dead-end cells where no tetracube fits without leaving a gap. Script retries up to 50 times automatically.

2D slab (reliable):



“fat” — always solvable

3D tendril (UNSAT):



verify.py — 6 independent checks

- 1 Exactly 8 pieces placed
- 2 Each piece has exactly 4 cells
- 3 All cells within the declared shape
- 4 No two pieces share a cell
- 5 Every shape cell is covered
- 6 Each piece type appears exactly once

Reconstructs and checks the solution *without trusting Clingo's output*.

Performance (all shapes solved in < 60 ms)

Shape	Time	Result
T-shape	< 50 ms	SAT
L-shape	< 50 ms	SAT
Plus/Cross	≈ 35 ms	SAT
Rnd. 2D slab	< 30 ms	SAT
Rnd. 3D	1–60 ms	varies

What we proved

- Non-rectangular 3D volumes **are** perfectly tileable by the 8 standard tetracubes
- A single `cell/3` predicate suffices to define *any* target volume for the solver
- The same ASP engine solves T, L, Plus/Cross, and random polycubes **without modification**
- All solutions pass 6 independent correctness checks

Thank you! Questions?